

Project-1 of “Neural Network and Deep Learning”

Yanwei Fu, Shufeng Nan

April 19, 2026

Abstract

(1) This is the first project of our course. The deadline is **11:59 PM, May 24, 2026**. Please submit your report on eLearning. Contact: **deeplearning.fudan@yandex.com** for any assistance.

(2) The goal of your write-up is to document the experiments you have done and your main findings. Please explain your results clearly. Hand in a single PDF file of your report. Enclose a Github link to your code in the submitted file. When uploading your code to Github, do not upload the dataset, model weights, or other large files. You should also provide a link to your trained model weights or saved checkpoints in your report. We recommend uploading model files to the ModelScope platform. Please also put your name and Student ID in your paper.

(3) About the deadline and penalty. In general, you should submit the paper according to the deadline of this mini-project. Late submission is also acceptable; however, you will be penalized **10%** of the score for each week’s delay.

(4) We provide Python starter code using the NumPy package as a general reference. You need to implement some functions and methods on your own. See more information in the README of the provided code. You may use other programming languages if you want, but you must implement your own version first.

(5) Note that the goal of this project is to practice the basic components of a neural network and a CNN. Therefore, do not invoke deep learning functions or modules that directly solve the required tasks (for example, PyTorch modules for the same operators). CPU is sufficient for this project; you do not need GPUs.

1 Neural Network

In this problem we will investigate handwritten digit classification on the MNIST dataset. MNIST contains 60,000 training images and 10,000 testing images. Each image is a 28×28 grayscale image and is labeled with the correct digit (0–9) it represents. You need to implement neural network models to recognize handwritten digits, conduct experiments to test your models, and explain the main observations from your results.

Compared with the previous version of this project, the emphasis is not on trying as many modifications as possible, but on building a solid baseline, implementing a CNN, and conducting clear and fair comparisons. After completing the basic requirements, you should further choose **two** additional directions to explore.

1.1 Project Overview

This project contains three parts:

1. **Part A: MLP Baseline (Required)**
2. **Part B: CNN Model (Required)**
3. **Part C: Choose Two Additional Directions**

You must only use the provided MNIST dataset. Do not use any external dataset.

1.2 Part A: MLP Baseline (Required)

Build and train a baseline Multi-Layer Perceptron (MLP) model for MNIST classification.

You are required to:

1. implement the forward and backward propagation of a linear layer;
2. implement the cross-entropy loss with softmax;
3. train an MLP on MNIST;
4. report the training and validation performance;
5. include at least one learning curve in your report.

1.3 Part B: CNN Model (Required)

Implement a simple CNN and compare it with the MLP baseline.

You are required to:

1. implement the `conv2D` operator by yourself;
2. build a simple CNN model for MNIST classification;
3. compare the CNN with the MLP under reasonable and fair settings;
4. discuss why the CNN performs better or worse than the MLP.

This part is a core requirement of the project and should reflect what you have learned from the CNN part of the course.

1.4 Part C: Choose Two Additional Directions

After completing Part A and Part B, choose **two** of the following directions. For each chosen direction, you should explain what you changed or analyzed, compare it with an appropriate baseline, and discuss your conclusion clearly.

1.4.1 Direction 1: Optimization

Try one optimization-related modification, for example:

- momentum;
- learning rate scheduling;
- different learning rate settings.

1.4.2 Direction 2: Regularization

Try one regularization-related modification, for example:

- L2 regularization;
- dropout;
- early stopping.

1.4.3 Direction 3: Data Augmentation

Apply light transformations to the training images, for example:

- small rotation;
- small translation;
- slight resizing.

Compare the performance with and without augmentation.

1.4.4 Direction 4: Robustness Analysis

Evaluate your trained model under small perturbations of the input images, for example:

- small rotation;
- translation;
- Gaussian noise.

The purpose is to examine whether the model is stable, not only accurate.

1.4.5 Direction 5: Error Analysis and Visualization

Analyze your model through one or more of the following:

- confusion matrix;
- misclassified examples;
- visualization of MLP weights or convolution kernels.

A Appendix

A.1 Grading

The project will be graded mainly according to the following aspects:

1. **Implementation correctness:** 35%
2. **CNN implementation and MLP-vs-CNN comparison:** 30%
3. **Quality of the two chosen directions:** 20%
4. **Discussion and report clarity:** 15%

A.2 Notes

1. You do not need to pursue the best possible accuracy.
2. **A clear comparison and a good explanation are more important than trying many disconnected ideas.**
3. **Please make sure that your code can be run** and that your report matches your implementation.
4. When uploading your code to Github, **do not include the dataset, trained weights, or other large files.** We recommend **uploading model files to ModelScope** and providing the corresponding link in your report.

A.3 Where to Modify the Starter Code

You are expected to work mainly inside the `codes/` folder. The following files are the most relevant.

A.3.1 Required Files

1. `codes/mynn/op.py`

You should modify this file for the core operators:

- `implement Linear.forward;`
- `implement Linear.backward;`
- `implement MultiCrossEntropyLoss;`
- `implement conv2D.`

2. `codes/mynn/models.py`

You should modify this file for model definitions:

- use `Model_MLP` as your baseline model;

- implement `Model_CNN`;
- if needed, you may slightly adjust the structure of `Model_MLP`.

3. `codes/test_train.py`

You should use this file to run your training experiments:

- choose the model to train;
- set hyperparameters such as learning rate, batch size, and number of epochs;
- define the training setting for your baseline and CNN experiments.

4. `codes/test_model.py`

You should use this file to evaluate a saved model on the test set.

A.3.2 Optional Files

5. `codes/mynn/optimizer.py`

Modify this file if you choose optimization as one of your two directions:

- implement `MomentGD`.

6. `codes/mynn/lr_scheduler.py`

Modify this file if you choose optimization and decide to use a learning rate scheduler:

- implement `MultiStepLR`;
- implement `ExponentialLR`.

7. `codes/mynn/runner.py`

You normally do not need to rewrite this file. However, if your own CNN or training pipeline requires small adjustments, you may modify it carefully.

8. `codes/weight_visualization.py`

You may use or modify this file if you choose visualization in Part C.

9. `codes/hyperparameter_search.py`

You may use this file for optional hyperparameter search or experiment management.

A.4 Recommended Workflow

We recommend the following order:

1. finish `Linear` and `MultiCrossEntropyLoss` in `codes/mynn/op.py`;
2. run and verify your MLP baseline through `codes/test_train.py`;

3. implement `conv2D` in `codes/mynn/op.py`;
4. implement `Model_CNN` in `codes/mynn/models.py`;
5. compare MLP and CNN under similar training settings;
6. complete two focused additional directions.

A.5 Experimental Principle

Try to change one major factor at a time.

For example:

1. when comparing MLP and CNN, keep the training setting as similar as possible;
2. when studying dropout or momentum, do not simultaneously change many other hyperparameters.

Your goal is not to try as many tricks as possible, but to make your conclusions clear and convincing.

A.6 Report Requirements

Your report should include the following sections:

1. **MLP baseline**
2. **CNN model and MLP-vs-CNN comparison**
3. **Two additional directions**
4. **Main results table**
5. **Detailed visualization**
6. **Discussion**

The detailed visualization may include one or more of the following:

- learning curve;
- confusion matrix;
- visualization of weights;
- visualization of convolution kernels.

In your discussion, you may answer questions such as:

1. Why is CNN more suitable than MLP for image classification?
2. Does the CNN improve validation or test accuracy?
3. Which two additional directions did you choose, and why?
4. Which modification or analysis is the most informative?
5. What kinds of samples are still hard for your model?